



Heterogeneous Multi-Robot Collaboration Using Language Models

Exploring different sizes and architectures

Rohit Joseph Mamutil



Abstract

Heterogeneous warehouse fleets typically include robots with different roles, such as carrier vehicles and picker agents, that must coordinate task choice, role synchronization, and recharging while sharing shelves, corridors, and charging stations. This creates a discrete multi-agent decision problem in which throughput depends not only on motion planning, but also on when agents should commit to tasks, wait for support, or allocate time to battery recovery.

This thesis evaluates whether off-the-shelf language models (LMs) can support high-level decision-making for heterogeneous multi-robot collaboration in the Battery-TA-RWARE task-assignment warehouse environment. The control loop summarizes the current environment state, queries an LM for a grounded macro-action, and validates the proposed action against environment-defined feasibility constraints before execution. The experiments compare model size, planning architecture, and prompt representation, using delivery throughput, LM call efficiency, and observed failure behaviour as the main evaluation criteria.

The available data show that, in the shelf-delivery experiments, shared-context planning produced more deliveries than centralized planning and natural-language prompting produced more deliveries than JavaScript Object Notation (JSON) prompting. In the call-aware experiments, a mid-sized local model (llama3.2:3b) produced the highest delivery total among the completed natural-language runs, while the completed JSON runs produced lower throughput and higher call cost than their natural-language counterparts. Across the evaluated settings, the control loop operated through grounded action identifiers, candidate shaping, and explicit validation before execution.

Faculty of Science and Technology

Uppsala University, Place of publication Uppsala

Supervisor: Xuezhi Niu Subject reader: Didem Gürdür Broo

Examiner: Ramanathan Thinniyam Srinivasan

Sammanfattning

I heterogena robotflottor i lager används ofta olika robottyper (t.ex. bärande automated guided vehicles (AGVs) och lastningsagenter) som måste samordna sina handlingar för att uppnå hög genomströmning samtidigt som säkerhetskrav, såsom batterinivåer, uppfylls. Att konstruera styrsystem för sådana multi-robot systems (MRSs) är svårt på grund av stora diskreta handlingsrymder, glesa belöningssignaler och behovet av robust koordination.

Detta examensarbete undersöker om färdigtränade language models (LMs) kan användas som beslutsfattare på hög nivå för samarbete mellan heterogena robotar i en lager-miljö med uppgiftsallokering och batteridynamik (Battery-TA-RWARE). Metoden sammanfattar det multi-agentiska tillståndet i naturligt språk, låter en LM välja en handling per agent och validerar sedan handlingarna mot miljöns tillåtna handlingsmasker.

Arbetet levererar en första utvärderingspipeline för LM-styrning av multi-agentisk lagerlogistik samt preliminära resultat som visar att en enkel språkbaserad LM-policy presterar avsevärt sämre än en domänheuristik, vilket motiverar fortsatt arbete med bättre struktur, grounding och koordination.

Acknowledgement I would like to thank my supervisor, Xuezhi Niu, for continuous guidance and constructive feedback throughout this thesis work. I also thank my reviewer, Didem Grdr Broo, and my examiner, Ramanathan Thinniyam Srinivasan, for their input on the direction and quality of the work. Finally, I am grateful to the developers and maintainers of the open-source environments and tools used in this project.

Contents

Acronyms	vi
1 Introduction	1
1.1 Research Questions	10
1.2 Contributions	10
1.3 Report Outline	11
2 Background and Related Work	12
2.1 Multi-Robot Task Planning Approaches	12
2.2 Multi-Robot Systems and Coordination	16
2.3 Warehouse Task Assignment Environments	17
2.4 Language-Model-Based Planning and Grounding	18
2.5 Why Study Model Size and Prompt Design?	18
3 Methodology	20
3.1 Overview	20
3.2 Environment: Battery-TA-RWARE	21
3.3 Language-Model Control Loop	22
3.4 Experiment Design and Comparison Axes	24
3.5 Scenarios, Models, and Metrics	25
3.6 Experimental Protocol	26
3.7 Delimitations	27
3.8 Ethical and Sustainability Considerations	27
4 Preliminary Results and Discussion	29

4.1	Result Scope and Objective Progression	29
4.2	Objective 1: Shelf-Delivery Experiments	29
4.2.1	Planning architecture	29
4.2.2	Prompt representation	30
4.3	Objective 2: Call-Aware Shelf-Delivery Experiments	30
4.3.1	Prompt representation	31
4.3.2	Adaptation to the changed objective	31
4.3.3	Model size	31
4.3.4	Call-efficiency outcome	32
4.4	Objective 3: Battery-Aware Objective	32
4.5	Discussion	32
4.5.1	RQ1: Model size	32
4.5.2	RQ2: Planning architecture	32
4.5.3	RQ3: Prompt representation	32
5	Conclusions and Future Work	34
5.1	Future Work	34
A	Appendix	42
A.1	Reproducibility: Running the Controllers	42
A.2	Prompt Excerpt (Illustrative)	43
A.3	Metric Definitions	43

Acronyms

AGV automated guided vehicle. ii, 16, 17, 21, 22, 25, 26

JSON JavaScript Object Notation. i, 9, 10, 25, 30–32, 34

LM language model. i, ii, 10, 11, 17, 18, 20, 22, 24, 25, 29–32, 34, 35, 43

MARL multi-agent reinforcement learning. 17, 35

MAS multi-agent system. 16

MRS multi-robot system. ii, 16

PPO proximal policy optimization. 35

TA-RWARE task-assignment robotic warehouse environment. 21

1 Introduction

Long before cities, agriculture, written records, or modern technology, humans lived in small groups navigating an uncertain world. Survival depended not only on individual intelligence but also on the ability to act collectively. Groups hunted, gathered food, protected offspring, shared resources, and learned from one another. Many accounts of human social evolution emphasise cooperation and social bonding as important pressures shaping communication [1, 2].

As human societies became larger and their activities more complex, the demands placed on communication also increased. Groups needed to remember where resources had previously been found, teach tool-making techniques, coordinate hunting strategies, divide responsibilities, and adapt plans when circumstances changed. Improved communication was needed to refer to distant locations, future actions, past experiences, hidden constraints, and the intentions of other individuals [3, 1].

Language likely emerged gradually alongside human social and cognitive evolution rather than appearing as a fully formed system [4]. Simple signals became richer, more structured, and more expressive. Sounds, gestures, and shared symbols acquired stable meanings that enabled more sophisticated interactions. As collective activities became more demanding, communication evolved to support them. At the same time, social and organisational pressures favoured further advances in communication [1, 2].

This co-evolution produced a feedback loop in which improved communication supported larger groups, specialised roles, more effective teaching, and advanced planning, while these social developments encouraged further refinements in communication. Many accounts of language evolution consequently view language and collective action as deeply intertwined phenomena. Language is often associated with shared intentionality, social organisation, cooperation, and the practical demands of acting together in complex environments [3, 1, 2, 4].

The development of written language further extended these capabilities. Knowledge, procedures, plans, and experiences could now persist beyond immediate interactions and be shared across larger groups and across generations [5]. Human societies accumulated large bodies of recorded information, creating new opportunities for learning, coordination, and collective problem-solving.

The growth of written language and, later, digital information created an unprecedented repository of human knowledge [5]. Books, articles, manuals, conversations, reports, and countless other forms of written communication captured facts, goals, plans, instructions, explanations, negotiations, and other artefacts of human problem-solving. As digital text collections expanded, researchers began developing computational methods

capable of modelling and generating language automatically [6].

Early computational language models represented language through statistical relationships between words and phrases [6]. Neural probabilistic language models later showed how such relationships could be learned through distributed representations [7], and word-embedding methods scaled distributed representations to large text corpora [8]. The introduction of transformer-based architectures marked a significant advance in language modelling [9]. Together, these developments contributed to the emergence of large-scale foundation models trained on broad data collections [10].

Modern Large Language Models (LLMs) are trained on extensive collections of human-generated language [11, 10]. Unlike systems based on manually encoded rules or task-specific reasoning procedures, they acquire behaviour from statistical patterns in examples contained within their training data [11, 10]. This exposes the models to many forms of human activity expressed through language, including planning, instruction following, explanation, negotiation, coordination, and collaborative problem-solving [11, 10].

Although LLMs are fundamentally trained to predict the next token in a sequence, recent studies suggest that they can exhibit capabilities extending beyond simple text completion [10, 6]. Under suitable prompting strategies, language models have been shown to generate plausible plans [12], decompose complex tasks into intermediate sub-goals [13], reason over sequences of actions [14], and explore alternative reasoning paths when constraints or objectives change [15]. These observations have motivated growing interest in the use of language models as components of decision-making and planning systems rather than solely as tools for text generation.

A natural question is whether, if language evolved alongside sophisticated forms of collective action and modern language models acquire patterns from language that humans use to describe goals, plans, responsibilities, and constraints, such models could contribute to decision-making in situations where multiple entities must work together towards a shared objective. The present thesis investigates this possibility in the context of multi-agent systems.

Many real-world problems cannot be solved effectively by a single decision-making entity. Instead, they involve multiple entities that must operate within a shared environment while pursuing individual or collective objectives. In artificial intelligence and robotics, such entities are commonly referred to as agents. An agent is a system that perceives its environment and selects actions based on those perceptions [16]. When multiple agents operate within the same environment, the resulting system is commonly referred to as a multi-agent system (MAS) [17].

As the number of agents increases, decision-making becomes more challenging. Agents

may compete for resources, interfere with one another, require access to shared information, or depend on each other to complete tasks. Consequently, the ability to organise and align the actions of multiple agents becomes a central concern in multi-agent and multi-robot systems [18].

Researchers have proposed several concepts to describe different forms of collective behaviour among agents. Terms such as coordination, cooperation, and collaboration are often used throughout the literature, although their meanings are not always consistent. In some cases the terms are used interchangeably, while in others they describe distinct levels of interaction between agents. Because these concepts are central to the present thesis, the following definitions are adopted.

In this thesis, *coordination* refers to arranging actions, timing, information, or resource usage so that agents do not interfere with one another and can make progress towards their objectives. *Cooperation* refers to situations in which agents pursue a shared objective, even if their individual actions are not tightly interdependent. *Collaboration* is used for the stronger case where agents depend on one another to complete a task and where progress requires explicit interaction between different roles or capabilities. These distinctions are important because the literature does not always use the terms consistently, particularly across multi-agent systems, multi-robot systems, and human teamwork [18].

A particularly important class of multi-agent systems is the multi-robot system (MRS), where the agents are physical robots operating within a shared environment. Multi-robot systems have been studied in domains such as manufacturing, logistics, exploration, transportation, and search-and-rescue operations [18]. As in human teams, effective performance often depends on the capabilities of individual agents and their ability to coordinate and collaborate with one another.

Multi-robot systems appear in many domains, including manufacturing, transportation, agriculture, exploration, and logistics. Among these, warehouse logistics has received particular attention due to the rapid growth of e-commerce and the increasing demand for fast order fulfilment. According to recent estimates, global retail e-commerce sales exceeded USD 6 trillion in 2024 and continue to grow steadily, while e-commerce activity now represents a significant share of global retail transactions [19, 20]. This growth places substantial pressure on warehouse operations, fulfilment networks, and inventory management systems. As a result, improving warehouse efficiency has become both an important industrial objective and an active area of research.

Warehouse environments are also particularly relevant from a multi-agent systems perspective. Fulfilling customer orders requires the interaction of multiple resources, including storage locations, transportation systems, workstations, inventory-management

processes, and, increasingly, autonomous robots. These resources operate under shared constraints and must be coordinated to ensure efficient order fulfilment, making warehouse logistics a practical and representative domain for studying multi-agent and multi-robot decision-making [21].

Consider a customer placing an online order. The requested item may be stored on a shelf among thousands of products distributed throughout a warehouse. To fulfil the order, the item must be located, retrieved from storage, transported to a picking or packing area, prepared for shipment, and eventually dispatched for delivery. Throughout this process, multiple resources must interact efficiently to ensure that the correct item reaches the customer on time [21].

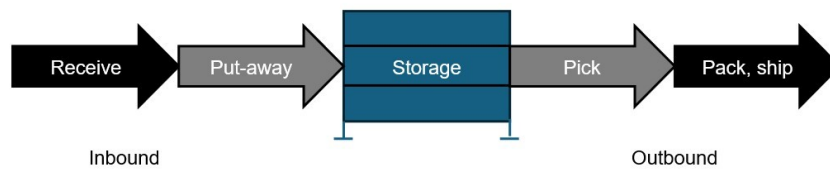


Figure 1 Overview of the warehouse goods flow

Warehousing is a broad discipline with its own terminology and operational practices. This thesis introduces only the concepts necessary to understand the experimental environment; a more comprehensive discussion of warehouse operations can be found in Bartholdi and Hackman [21].

In a typical warehouse, products are stored in designated storage locations such as shelves, racks, bins, or movable storage units. Customer requests are represented as orders that specify which items must be retrieved. The process of retrieving requested items from storage is commonly referred to as picking. Depending on the warehouse design, picked items may be transported to packing stations, where orders are prepared for shipment, before being sent to delivery or dispatch stations for onward transportation [21, 22].

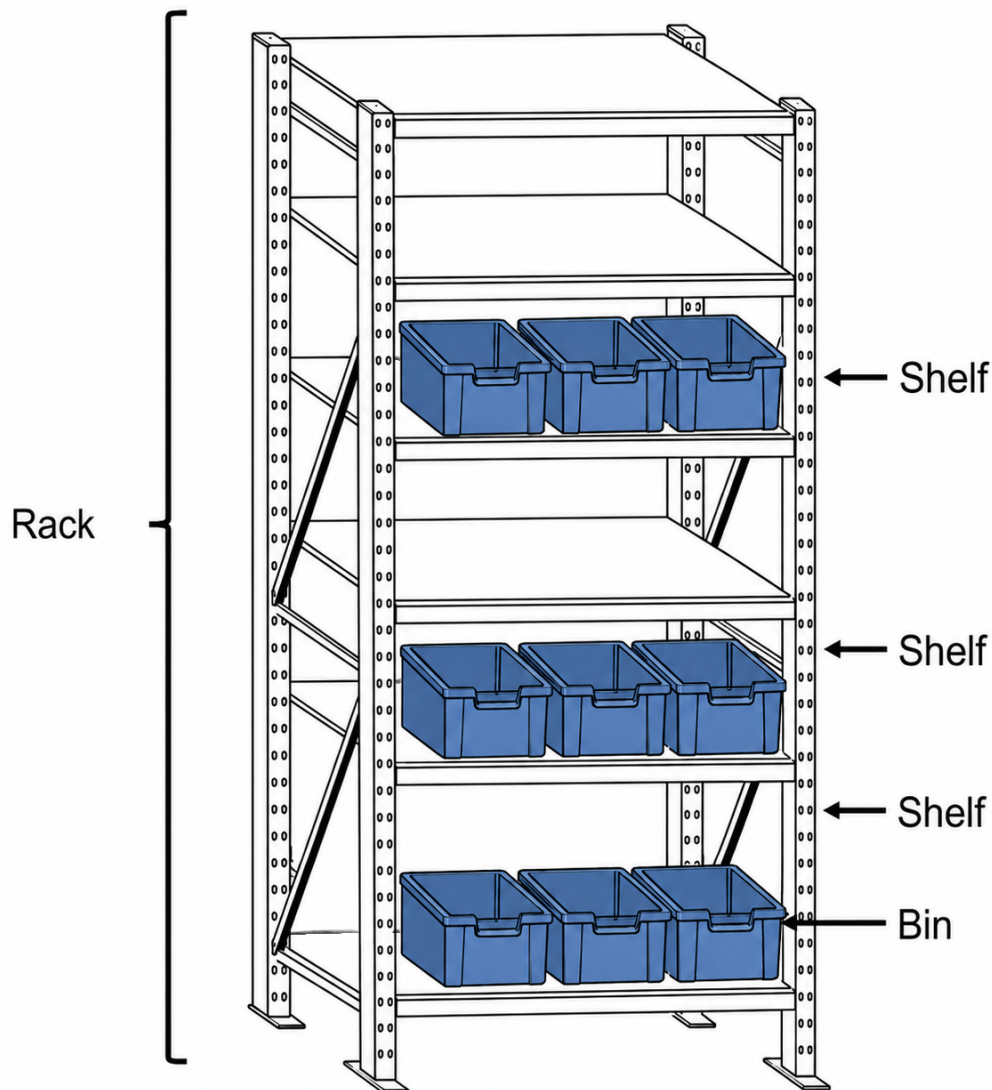


Figure 2 Example warehouse storage hierarchy showing a rack composed of multiple shelves and storage bins.

The resources involved in these activities may include human workers, automated storage systems, mobile robots, workstations, and warehouse management software. Together, these components form a workflow whose objective is to fulfil customer orders accurately and efficiently. Traditionally, warehouses operated according to a person-to-goods paradigm. Human workers travelled through storage aisles to locate and retrieve products. While straightforward to implement, worker travel often represented a signif-

icant portion of fulfilment time and operational cost [22].

As order volumes increased, particularly with the growth of e-commerce, alternative approaches emerged. In goods-to-person systems, products or storage units are brought to stationary workers instead of requiring workers to travel throughout the warehouse. This reduces travel time but introduces new coordination challenges, since storage units, workstations, and transportation resources must now be scheduled and synchronised effectively [23, 24].

Robotic warehouse systems emerged as a natural extension of this evolution. A well-known example is the Kiva-style warehouse system, in which fleets of mobile robots transport inventory pods between storage areas and stationary workstations [25]. Unlike traditional racks, which remain fixed in place, inventory pods are movable storage units containing multiple shelves and stored products. Mobile robots navigate beneath these pods, lift them, and transport them to workstations where items can be retrieved [25].

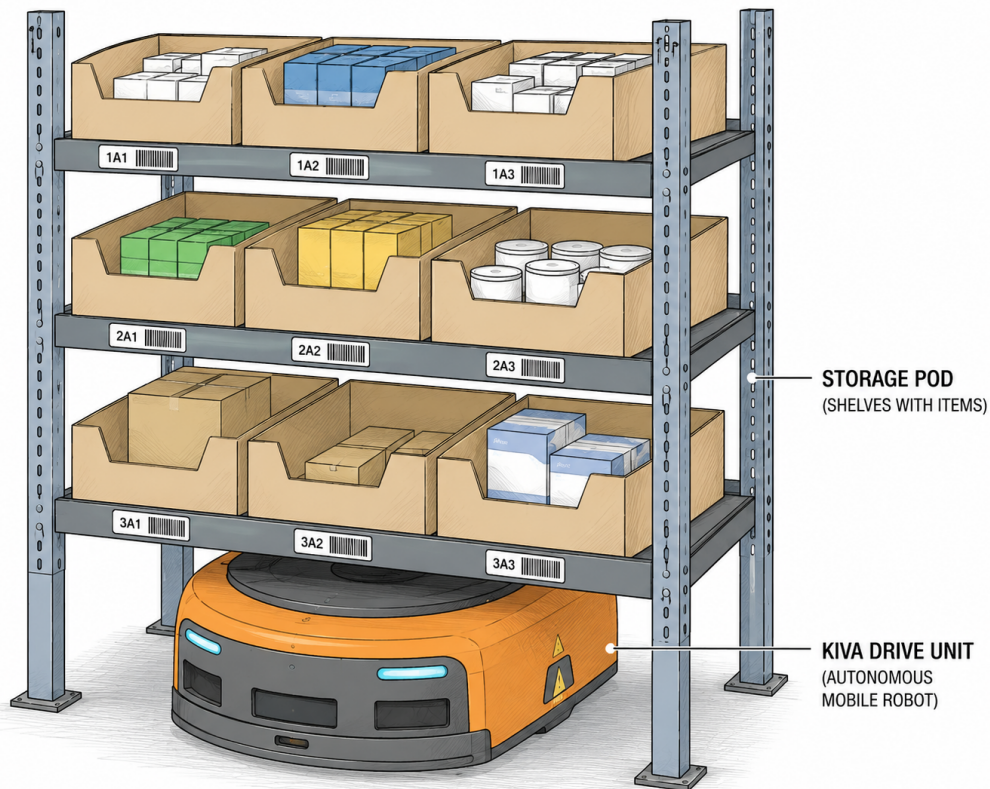


Figure 3 Illustration of a Kiva-style inventory pod. The pod contains multiple shelves used for product storage, while a mobile drive unit operates beneath the pod to transport it throughout the warehouse [25].

Rather than asking workers to travel to products, the warehouse brings products to workers. By transporting storage units to stationary workstations, goods-to-person(GTP) systems reduce the need for worker travel and can improve fulfilment efficiency [25]. At the same time, they introduce additional coordination requirements among robots, storage units, and workstations. The challenge is no longer limited to locating products and fulfilling orders. The warehouse must now determine which storage unit should be moved, which robot should perform the movement, which workstation should receive the requested items, and how shared resources should be allocated when multiple requests compete for attention. As the number of robots, workstations, and orders increases, these decisions become increasingly interdependent [25, 24].

At this point, warehouse operation becomes a coordination problem. Robots may compete for pathways, storage units, charging stations, or workstations. Tasks may depend on the completion of other tasks. Some activities can be performed independently, while others require multiple resources or specialised roles to interact successfully. The overall performance of the warehouse depends on the capabilities of individual agents and the effectiveness of their coordination [18]. Although warehouse logistics provides a useful example, similar challenges appear throughout multi-agent systems. Autonomous vehicle fleets, drone swarms, manufacturing systems, search-and-rescue teams, and service robots must all operate under shared constraints while pursuing common objectives. The underlying challenge is therefore more general than warehouse automation: how can multiple agents make effective decisions while acting within a shared environment?

While coordination is essential in robotic warehouses, many real-world multi-agent systems require a stronger form of interaction. Coordination may be sufficient when agents perform largely independent tasks while sharing resources. Collaboration, however, requires agents to depend on one another in order to achieve a common objective. In such settings, successful task completion depends on the interaction of agents with complementary roles or capabilities [18, 26].

Collaborative decision-making introduces additional challenges. Decisions made by one agent may influence the opportunities available to others, while task completion may depend on actions performed by multiple agents. Effective behaviour therefore requires consideration of shared goals, task dependencies, resource constraints, and the expected behaviour of teammates [26]. These characteristics make collaborative multi-agent systems an interesting domain for investigating whether language-model-based planning and decision-making can provide useful support.

In many practical multi-agent systems, this interdependence arises because agents possess complementary capabilities rather than identical roles. Progress may therefore depend on interactions between specialised agents that must work together to complete

tasks.

Evaluating such decision-making approaches directly in physical robotic systems can be costly, difficult to reproduce, and potentially disruptive to ongoing operations. Changes to coordination or collaboration strategies may affect safety, productivity, equipment utilisation, and interactions between agents operating within the environment. Consequently, simulation has become a widely adopted methodology for studying multi-agent and multi-robot systems [27, 28].

Simulation environments provide controlled and repeatable conditions in which task distributions, agent capabilities, resource constraints, and environmental characteristics can be varied systematically. This enables researchers to compare different approaches under identical conditions before considering deployment in physical systems.

The growing adoption of simulation has led to the development of standardised benchmark environments. Frameworks such as OpenAI Gym and its successor Gymnasium provide common interfaces that support reproducible experimentation across a wide range of reinforcement-learning and decision-making problems [29, 30]. Warehouse-inspired benchmark environments have become particularly popular because they capture many of the coordination and collaboration challenges encountered in real-world systems. Agents must share resources, resolve conflicts, allocate tasks, and operate towards common objectives while acting under environmental constraints. Although simplified relative to real warehouses, such environments provide a useful platform for studying multi-agent decision-making [28].

Many approaches have been proposed for decision-making in multi-agent and multi-robot systems. Depending on the application, researchers have employed optimisation methods, heuristic approaches, rule-based control, task-allocation algorithms, and more recently multi-agent reinforcement learning. These techniques have demonstrated success across a variety of coordination and planning problems, but they may require domain-specific design, extensive training, reward engineering, or adaptation when objectives and environmental conditions change [31, 32, 27].

Recent advances in large language models have motivated interest in their use beyond traditional text-generation tasks. Their demonstrated ability to generate plans, reason about constraints, decompose tasks, and interpret structured descriptions of environments has led researchers to explore their use in planning and robotics applications [12, 33, 34]. These developments raise the possibility that language models could serve as high-level decision-making components in collaborative multi-agent systems.

Despite growing interest in language-model-based planning, relatively little work has investigated their use for collaborative decision-making among heterogeneous agents operating under shared constraints. Existing studies often focus on single-agent tasks,

embodied planning for individual robots, or coordination settings in which agents perform largely independent activities [12, 33, 35]. Less attention has been given to environments where successful task completion depends on interactions between specialised roles and where decisions must remain grounded in a constrained action space.

Several practical questions therefore remain open. Does increasing model size improve collaborative performance? How should planning responsibilities be distributed when multiple agents require decisions? Does the representation used to describe the environment influence the quality of the resulting decisions? Addressing these questions is important for understanding whether language models can contribute to collaborative multi-agent systems and how such systems should be designed and deployed in practice.

To investigate these questions, the present thesis evaluates language models as high-level decision-making components within a collaborative multi-agent setting. Rather than focusing on low-level robot control, the study examines how different design choices influence collaborative task performance. In particular, three factors are considered: the size of the language model, the architecture used for planning, and the representation used to describe the environment state.

One important design consideration concerns the planning architecture. In a *centralized planning* approach, a single prompt receives a shared description of the environment and returns decisions for all agents requiring planning. An alternative approach is *shared-context per-agent planning*, where agents are planned individually but each prompt still contains a summary of the shared environment state. These approaches represent different ways of distributing decision-making responsibility while maintaining access to global context.

Another important consideration concerns how information is presented to the language model. Software and robotic systems commonly exchange information using structured data representations rather than natural language. One widely adopted format is JavaScript Object Notation (JavaScript Object Notation (JSON)), which represents system state through structured key-value pairs and hierarchical data structures. Such representations are convenient for software integration because they can often be generated directly from program variables with minimal transformation.

However, language models are primarily trained on human-generated text and may therefore process information differently depending on how information is represented. Recent studies suggest that prompt structure and information representation can influence language-model behaviour and task performance, while structured formats may affect reasoning performance in some settings [36, 37]. This creates an important practical question for engineering applications. Should system state be provided directly in a structured format such as JSON, or is it beneficial to invest additional effort in

transforming the same information into natural-language descriptions that more closely resemble the data on which language models were trained? The present thesis investigates this question alongside the effects of model size and planning architecture.

1.1 Research Questions

The thesis is guided by the following main research question:

Main Research Question: To what extent can prompted language models (LMs) support grounded high-level decision-making for heterogeneous multi-robot collaboration in a constrained warehouse environment?

To investigate this question systematically, the study examines the influence of three design factors: language-model size, planning architecture, and prompt representation. The following sub-questions are considered:

1. **RQ1 (Model Size):** How does language-model size affect collaborative task performance in a constrained warehouse environment, measured through delivery throughput, language-model call efficiency, and observed failure behaviour?
2. **RQ2 (Planning Architecture):** How do centralized planning and shared-context per-agent planning differ in their ability to support collaborative task performance, measured through delivery throughput, robustness across scenarios with varying collaboration demands, and the prevalence of non-productive or failed runs?
3. **RQ3 (Prompt Representation):** How do natural-language and JSON-based environment representations affect collaborative task performance, measured through delivery throughput, language-model call efficiency, and the consistency of valid and productive decisions?

The research questions are evaluated in a warehouse-inspired collaborative environment involving heterogeneous agents operating under shared constraints. Performance is assessed using delivery throughput, language-model call efficiency, failure behaviour, and scenario-specific robustness measures.

1.2 Contributions

The current thesis work contributes:

- A grounded evaluation framework for studying language-model-based high-level decision-making in collaborative heterogeneous multi-robot systems. The approach constrains language-model outputs to environment-defined actions and validates proposed decisions against explicit feasibility constraints.
- Comparative empirical evidence on the effects of model size, planning architecture, and prompt representation in language-model-based collaborative decision-making. The results characterise their impact on throughput, language-model utilisation, robustness, and failure behaviour, while providing insights into the role of grounding and validation mechanisms.

1.3 Report Outline

Section 2 provides background on multi-robot coordination, warehouse task assignment, and LM-based planning. Section 3 describes the environment, the proposed LM control loop, the experimental design, and the main delimitations of the study. Section 4 presents the current experimental results together with their interpretation. Finally, Section 5 concludes and outlines future work.

2 Background and Related Work

This chapter reviews the literature relevant to language-model-based task planning in heterogeneous multi-robot systems. It examines the evolution of task planning approaches, recent advances in prompt design and grounding techniques, considerations in language model selection and deployment, and the simulation environments used to evaluate multi-robot systems. Together, these developments establish the current state of the field and highlight the research gaps addressed in this thesis.

2.1 Multi-Robot Task Planning Approaches

Task planning is one of the fundamental problems in multi-robot systems, concerned with determining how tasks should be allocated, ordered, and coordinated among multiple robots to achieve a shared objective efficiently. Unlike single-robot planning, where decisions primarily concern the actions of an individual agent, multi-robot task planning must additionally account for interactions between robots, including resource sharing, task dependencies, communication constraints, and conflicting objectives. As the number of robots and environmental complexity increase, the planning problem becomes increasingly challenging due to the combinatorial growth of possible task allocations and execution sequences. Consequently, designing planning strategies that remain both computationally tractable and capable of coordinating heterogeneous robots has been a long-standing research challenge.

Early research addressed these challenges through explicitly engineered planning and coordination strategies. Rather than learning behaviours from experience, robots followed predefined rules, optimisation procedures, or heuristic decision-making policies designed for specific tasks and environments. Gerkey and Mataric [31] formalised the Multi-Robot Task Allocation (MRTA) problem and proposed one of the earliest taxonomies for categorising task allocation approaches according to robot capabilities, task characteristics, and allocation strategies. Their work established a theoretical foundation for analysing and comparing multi-robot planning algorithms and continues to underpin subsequent research in task allocation.

Building upon these developments, Yan *et al.* [18] reviewed the evolution of multi-robot coordination, highlighting a wide range of planning approaches including auction-based task allocation, cooperative transport, communication-aware coordination, motion planning, and early learning-based methods. Representative systems such as MURDOCH demonstrated that auction mechanisms could efficiently allocate tasks among heterogeneous robots while maintaining fault-tolerant coordination through distributed bidding.

Other approaches addressed tightly coupled cooperative tasks by explicitly considering robot availability, spatial constraints, and temporal dependencies when assigning sub-tasks to multiple robots. Collectively, these studies demonstrated that effective multi-robot task planning requires considerably more than assigning tasks to individual robots; successful execution also depends on communication, coordination, conflict resolution, and synchronisation among cooperating agents.

Although heuristic and optimisation-based approaches proved effective in structured environments with well-defined objectives, they generally relied on handcrafted decision rules, explicit environment models, and carefully designed cost functions. As task complexity and environmental uncertainty increased, adapting these systems to new scenarios often required substantial manual redesign rather than simple reconfiguration. These limitations motivated growing interest in learning-based approaches capable of acquiring coordination strategies directly through interaction with the environment.

The limitations of heuristic and optimisation-based planning motivated a growing shift towards learning-based approaches, particularly Reinforcement Learning (RL), in which agents learn decision-making policies through repeated interaction with the environment rather than relying on predefined rules or handcrafted optimisation objectives. Extending this paradigm to collaborative robotics, Multi-Agent Reinforcement Learning (MARL) enables multiple agents to simultaneously learn cooperative behaviours while adapting to both environmental dynamics and the actions of neighbouring agents. Compared with traditional planning methods, MARL offers greater flexibility in dynamic environments, allowing coordination strategies to emerge through experience instead of explicit programming. Consequently, MARL has become one of the dominant approaches for solving cooperative decision-making problems in warehouse logistics, autonomous transportation, and other multi-robot applications.

As MARL research rapidly expanded, numerous algorithmic paradigms were proposed, including independent learning, centralised policy-gradient methods, and value decomposition approaches. However, the absence of common benchmark environments and evaluation methodologies made objective comparison between algorithms increasingly difficult. Papoudakis *et al.* [27] addressed this challenge through one of the first comprehensive benchmarking studies of deep MARL algorithms across a diverse collection of cooperative tasks. Their evaluation compared representative algorithms from the major classes of MARL methods under consistent experimental conditions and demonstrated that no single learning paradigm consistently outperformed the others across all environments. The study further highlighted that algorithm performance is strongly influenced by factors such as reward sparsity, coordination complexity, and parameter sharing. To encourage reproducible research, the authors introduced the EPyMARL framework together with additional benchmark environments designed to evaluate cooperative coordination under sparse reward settings. This work established an important experimental

foundation for subsequent MARL research by promoting standardised evaluation practices and facilitating fair comparison between learning algorithms.

Building upon these developments, research increasingly focused on applying MARL to realistic industrial problems. Krnjaic *et al.* [28] investigated the order-picking problem in heterogeneous warehouse environments involving autonomous mobile robots and human workers. The authors observed that existing warehouse management systems rely heavily on handcrafted heuristics that require significant engineering effort and must often be redesigned for different warehouse layouts and operational requirements. To address these limitations, they proposed a hierarchical MARL architecture in which a manager agent assigns high-level goals to worker agents while both manager and worker policies are jointly optimised towards maximising warehouse throughput. By decomposing decision making into hierarchical levels, the proposed framework substantially reduces the action space explored by worker agents, resulting in improved sample efficiency and faster policy learning. Experimental evaluation demonstrated higher warehouse throughput than several widely used industrial heuristics as well as conventional MARL baselines across diverse warehouse configurations. In addition to the proposed learning architecture, the authors introduced the Task Assignment Robotic Warehouse (TA-RWARE) benchmark, extending the RWARE environment to support heterogeneous task assignment and providing a reproducible platform for evaluating high-level coordination strategies.

While hierarchical MARL demonstrated that complex warehouse coordination could be learned effectively, subsequent research began investigating how cooperation itself could be improved. Rather than modifying network architectures alone, researchers explored whether richer learning objectives could encourage more effective collaborative behaviour among heterogeneous robots. Niu and Grdr Broo [38] proposed a biologically inspired reward shaping framework based on ecological symbiosis. Instead of rewarding agents solely for their individual task performance, the framework explicitly models mutually beneficial interactions between robots, encouraging behaviours that improve collective performance. The authors argue that conventional MARL frequently suffers from sparse reward signals, unstable convergence, and difficult credit assignment in cooperative tasks. Through experiments involving heterogeneous robotic systems, they demonstrated that incorporating symbiotic relationships into the reward function improved convergence stability, reduced learning variance, and promoted more effective cooperation, particularly as task complexity increased. Their work highlights that reward design itself plays a critical role in determining the quality of learned coordination.

Extending these ideas to warehouse logistics, Niu *et al.* [39] integrated ecological symbiosis principles into a Centralised Training with Decentralised Execution (CTDE) MARL framework for heterogeneous multi-robot systems. Inspired by resource-sharing mech-

anisms observed in natural ecosystems, the proposed architecture enables robots to exchange critical state information, including battery levels and resource availability, allowing agents to dynamically coordinate charging behaviour and workload distribution. Evaluated within a simulated warehouse environment, the symbiotic MARL framework achieved improvements in overall task performance, resource utilisation, and balanced energy consumption compared with conventional MARL approaches. Beyond demonstrating improved coordination, the study illustrates how explicitly modelling inter-agent dependencies can increase system resilience and adaptability under changing operating conditions.

Collectively, these studies demonstrate that learning-based approaches have substantially advanced heterogeneous multi-robot coordination. However, they remain dependent on environment-specific training, carefully engineered reward functions, and computationally intensive policy optimisation. Furthermore, adapting learned policies to new environments often requires retraining. Consequently, recent research has begun exploring pretrained foundation models, particularly Large Language Models (LLMs), as an alternative paradigm for high-level task planning. Rather than learning task-specific policies from scratch, LLMs leverage knowledge acquired during pretraining to reason about tasks and generate planning decisions from natural language or structured environment descriptions.

Recent surveys classify LLM applications in robotics according to the level of decision making they support, including high-level task planning and task allocation, mid-level motion planning, and low-level action generation [40]. High-level planning concerns reasoning over task objectives and coordination, whereas lower levels focus on trajectory generation or direct robot control.

Existing LLM-based planning frameworks commonly employ language models as high-level decision makers while retaining conventional perception, navigation, and control modules. SayCan grounds language-model action selection with affordance estimates so that plans are biased towards actions that are both useful for the instruction and feasible in the current robotic state [33]. Inner Monologue extends this pattern by incorporating feedback from the environment into language-model planning, while Prog-Prompt represents situated task plans as program-like structures over available robot actions [41, 42]. Code as Policies instead uses the language model to generate executable programs over predefined robot APIs, making code the intermediate policy representation between language instructions and embodied control [34]. Related zero-shot planning work shows that language models can propose action sequences from textual task descriptions when they are given an explicit action interface, although such plans still require grounding, validation, and replanning when deployed in physical or simulated environments [12].

2.2 Multi-Robot Systems and Coordination

multi-robot systems (MRSs) study how multiple embodied agents share resources, divide work, communicate, and avoid conflicts while pursuing a common objective. The broader field of multi-agent system (MAS) can also include purely software agents, whereas MRSs typically emphasize physical robots operating under motion, sensing, and actuation constraints. Classical overviews of cooperative robotics highlight recurring concerns such as group architectures, resource conflicts, and learning [43]. Survey treatments vary in terminology and scope (e.g., differences in how MAS and MRS are framed, and how communication and coordination taxonomies are defined) [18, 44]. In heterogeneous teams, these concerns intensify because roles and capabilities differ, and success often requires explicit synchronization, such as meeting at the same physical location at the same time [45]. This is directly relevant to the thesis, where automated guided vehicles (AGVs) and pickers cannot complete the full warehouse cycle independently and therefore depend on timely coordination under shared-space and battery constraints.

One useful way to organize prior work is by the aspects of coordination it emphasizes. Some work focuses on *coordination structure*, for example whether decisions are made centrally or through repeated local coordination. Other work focuses on *information sharing*, such as the role of explicit communication versus coordination through shared state or environment cues. Another recurring distinction is *decision timing*: whether coordination is fixed in advance, recomputed online, or revised in response to new state information. These dimensions map directly to the thesis design choices, especially the comparison between centralized and shared-context planning, the use of shared state summaries in prompts, and the use of step-synchronous conditional replanning rather than a fully fixed policy.

In this thesis, *collaboration* refers to interdependent task execution among heterogeneous roles, where agents must coordinate to complete an operation (e.g., a load/unload interaction). *Coordination* refers to mechanisms that prevent conflicts and deadlocks (e.g., collision avoidance and congestion handling), and *cooperation* refers to pursuing a shared objective even when agents may act more independently. These terms are not used consistently across communities, and the scope of MAS often extends beyond embodied robotics [26]. To reduce ambiguity, the thesis primarily uses MRS when referring to embodied robots and uses MAS to denote the broader class of multi-agent systems.

Coordination strategies can be described as *static* (offline, convention-based) or *dynamic* (online, reactive), depending on whether coordination rules are established prior to execution or synthesized during execution from sensed/communicated information [18].

Communication can likewise be categorized as *implicit* (information inferred via sensing or environment traces) versus *explicit* (direct message passing), trading off scalability against accuracy and bandwidth [18]. For example, Zhang et al. study coordinated multi-agent reinforcement learning (MARL) under communication constraints, illustrating the broader trade-off between tighter coordination and limited information bandwidth in multi-agent decision-making [46].

Planning is a central lens for collaboration: multi-robot planning is often discussed as task planning (who does what) and motion planning (how to move), with optimal formulations typically being NP-hard and thus motivating approximate methods and heuristics [18]. In particular, multi-robot task planning includes task decomposition (splitting a mission into subtasks) and task allocation (assigning subtasks to agents), with much of the literature focusing on allocation due to tractability and benchmarking convenience [18, 31]. This observation aligns with the thesis hypothesis: LMs may be useful as a structured decision module for decomposition and allocation when paired with explicit grounding, feasibility constraints, and environment feedback.

2.3 Warehouse Task Assignment Environments

This thesis uses Battery-TA-RWARE, an adaptation of the Task-Assignment Robotic Warehouse environment introduced in [28]. It is used here as a structured warehouse testbed in which repeated shelf-delivery cycles create explicit coordination pressure across heterogeneous roles. The environment models two heterogeneous agent types:

- **AGVs:** carrier agents that transport shelves between storage locations and goal stations.
- **Pickers:** support agents that must meet AGVs at shelf locations for loading and unloading.

Battery-TA-RWARE extends the task-assignment setting with battery depletion and charging stations, forcing agents to balance throughput with energy safety. Low-level movement is handled by environment dynamics (including A* path planning), while the controller selects discrete targets representing shelves, goals, or charging stations. This makes the environment especially useful for the thesis because it isolates the task-level coordination problem: which target should be chosen next, under what shared-state conditions, and with what timing relative to other agents.

2.4 Language-Model-Based Planning and Grounding

LMs can produce coherent natural-language plans, but robot control requires converting language into *grounded, feasible* actions. One influential pattern is to constrain the LM to select among validated, environment-defined options. For example, SayCan combines language-level reasoning with an affordance model to filter actions that are executable in the current state [33]. Other work frames LMs as planners that can propose action sequences given a symbolic interface to the environment, often coupled with explicit checking and replanning [12]. In language-only settings, prompting patterns such as interleaving reasoning and actions have been studied to improve decision quality under tool or environment feedback [14]. Recent multi-robot systems work explores *multi-agent* prompting where each robot maintains a separate dialog context and coordination emerges through structured communication plus environment feedback (e.g., collision checking and in-context plan revision) [35]. Recent work argues that closed-loop state feedback and a separation between high-level planning and low-level control can reduce hallucinated actions and improve long-horizon goal satisfaction in robot task planning with LMs [47]. In multi-robot task planning contexts, prior thesis work also emphasizes the need for a symbolic abstraction layer that translates sensor/state representations into language-level predicates that LMs can reliably consume [48].

These ideas motivate the design in this thesis: use a prompt that provides a compact but explicit mapping from environment state to action identifiers, require outputs to be parseable into discrete actions, and validate them against the environment’s current action constraints.

2.5 Why Study Model Size and Prompt Design?

In applied robotics and embedded contexts, compute, memory, energy, and latency constraints may favour smaller models for real-time deployment [49]. However, smaller models may require more structured prompting, tighter candidate sets, and stronger validation to reach acceptable performance. Recent robotics work explicitly motivates prompt design as a lever for compensating limited model capacity and improving decision quality in constrained deployments [50, 35]. At the same time, studies of prompt and output-format restrictions suggest that imposing rigid formats can affect language-model performance, which makes the choice between natural-language and structured representations an experimental variable rather than a purely implementation-level detail [36, 37]. In the warehouse setting of this thesis, these choices are not treated as generic language-model properties. They are studied because they are expected to affect whether generated actions remain feasible, whether coordination remains productive un-

der shared constraints, and how often the controller falls into invalid or non-productive behaviour. By evaluating multiple model sizes and prompt variants under the same environment and metrics, the thesis aims to clarify which components (model capacity, prompt structure, validation) most strongly affect feasibility, coordination quality, and throughput.

3 Methodology

3.1 Overview

This chapter serves two roles. It describes the implemented system, but it also justifies the empirical design used to answer the research questions. The thesis is therefore structured as a comparative experimental study: the same warehouse environment, evaluation metrics, and scenario catalogue are reused while selected design variables are changed in order to study their effect on collaborative task performance.

This design is appropriate for the thesis because the research questions are comparative rather than purely descriptive. RQ1 asks how model size affects performance, RQ2 asks how planning architecture affects performance, and RQ3 asks how prompt representation affects performance. These questions require the report to make explicit what is fixed, what is varied, and how the experiments are structured across the objective families.

The thesis uses an empirical, experimental methodology:

1. Implement an LM-driven control loop for a simulated heterogeneous warehouse environment.
2. Define the main comparison axes and evaluation metrics.
3. Run controlled experiments (fixed environment ids and seeds) across multiple LM models, planning architectures, and prompt variants.
4. Analyze throughput, safety, and collaborative task outcomes as well as qualitative failure modes.

What is fixed and what is varied. Across the main experiment families, the environment, action space, scenario catalogue, evaluation metrics, grounding/validation scheme, and local-inference setup are held as stable as practical. The principal variables are model size, planning architecture, prompt representation, and objective family. The report does not use an external heuristic baseline in the current comparison set, so the reported comparisons are made within the language-model-based control framework.

Interpretation and controls. The experiments are controlled in the sense that scenario ids, seeds, and runner configurations are recorded and reused within each fam-

ily. The surrounding validation layer filters infeasible outputs before execution, and the same comparison axes are used across the objective families.

3.2 Environment: Battery-TA-RWARE

Battery-TA-RWARE is a discrete-time, grid-based warehouse simulation derived from task-assignment robotic warehouse environment (TA-RWARE) [28]. At any time, a set of shelves is requested. Carrier AGVs must transport requested shelves to goal locations and then return them to storage. Picker agents are required to meet AGVs at shelf locations for load and unload interactions, creating explicit heterogeneous coordination constraints. Agents additionally have a battery state that decreases with movement and interactions; charging stations are available at fixed locations.

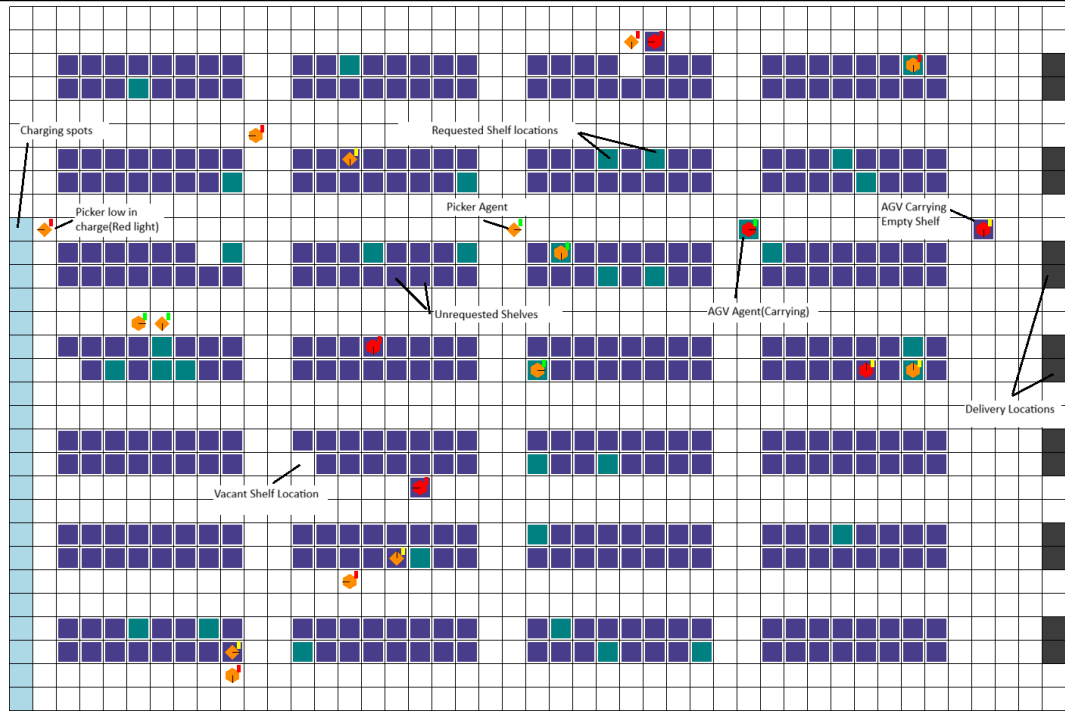


Figure 4 Illustration of the task-assignment warehouse setting used in this thesis.

The environment exposes a discrete action space where each action corresponds to selecting a target location:

- **Shelf actions:** choose a shelf location (e.g., to pick up or return a shelf).

- **Goal actions:** choose a delivery station (primarily relevant when an AGV carries a requested shelf).
- **Charging actions:** choose a charging station to recover battery.

Low-level motion toward the selected target is handled internally by the environment (A* path planning and collision resolution), while the controller is evaluated on the quality of these high-level decisions.

3.3 Language-Model Control Loop

The LM policy is implemented as a text-in / discrete-action-out controller:

1. **State summarization:** extract salient features of the global state (requested shelves and their action ids, per-agent position/target/battery/carrying status, charging occupancy).
2. **Candidate action set:** compute a valid-action mask from the environment and derive the prompt-visible candidate set from those feasible actions. These candidates are presented as compact role-relevant action rows with action ids and guidance fields such as distance, tags, and charging-related instructions.
3. **Prompting:** generate a per-agent prompt describing the agent role and the current state, including explicit mappings from requested objects to action ids.
4. **Querying:** send the prompt to a local inference server (Ollama) and obtain a text response.
5. **Parsing:** parse an integer action id (and optionally an action hold duration) from the response.
6. **Validation:** validate the parsed action id against the current valid-action mask. If the proposed action is missing or invalid, the controller discards it and falls back to an idle action for that step; in the action-persistence setting, an invalid persisted action clears the hold and forces a new query.
7. **Execution:** apply the validated joint action in the environment for one step (or hold the action for multiple steps, depending on configuration).

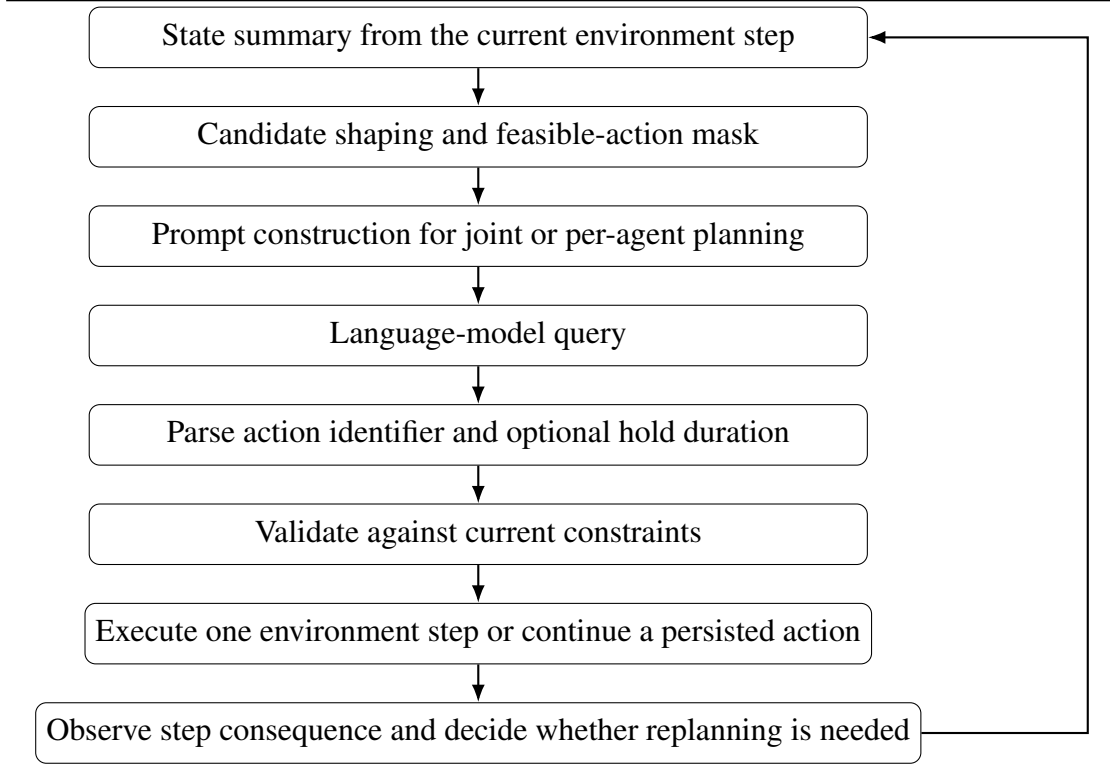


Figure 5 Schematic of the language-model control loop used in the experiments.

The controller is best described as a discrete-time, step-synchronous supervisory loop with conditional replanning. The environment advances one simulation step at a time, and at each step the controller reevaluates agent state and action validity. However, this does not imply that every agent is queried at every step. Instead, the experiments use two related control schemes.

- **Step-synchronous replanning with environment-owned continuation:** agents that are currently available for planning may receive a new high-level action, while busy agents are not replanned and continue their current task until the environment releases them.
- **Step-synchronous replanning with action persistence and conditional override:** accepted actions may remain active for multiple future steps, so agents are not re-queried while the persisted action remains valid. Replanning is triggered when a persisted action expires, becomes invalid, or when the controller decides intervention is needed, including selected busy-agent cases.

This design sits between two extremes. A fully scheduled scheme that queries every

agent at every step would generate many unnecessary LM calls, while a purely event-triggered scheme is less natural in this setting because useful triggers such as blocked progress, invalid persisted actions, expiring commitments, or changing support needs are only known after the system state is checked. The adopted design therefore combines regular per-step observability with selective replanning. This is also consistent with practical supervisory control in warehouse systems, where the fleet is typically checked at regular intervals, ongoing tasks continue by default, and new task-level decisions are issued only when the current situation justifies intervention.

Two design choices are central for feasibility:

- **Grounding via action ids:** the LM does not output coordinates or free-form text commands; instead it selects from discrete action ids that are already defined by the environment.
- **Hard validation:** action masks enforce constraints such as invalid goals for picker agents and busy-state restrictions. Invalid LM outputs are never executed.

The reported behaviour is therefore the behaviour of the complete control framework: prompt design, model output, candidate shaping, validation logic, and replanning policy act together inside the same supervisory loop.

3.4 Experiment Design and Comparison Axes

The experiments are organized as a controlled comparison within the grounded language-model control loop described above. Across the experiment families, the environment, discrete action space, validation layer, scenario catalogue, local-inference setup, and evaluation metrics are kept as stable as practical. The main factors that change are the objective formulation, model size, planning architecture, and prompt representation. This structure is important because the thesis does not only ask whether an LM can control the warehouse agents; it asks how different ways of using the LM affect collaborative task performance under comparable conditions.

The objective families define the main progression of the experimental campaign. The first family uses a delivery-only objective, where the controller is evaluated primarily on raw shelf-delivery throughput. This establishes the baseline interpretation for the later experiments: the controller is expected to select high-level actions that move requested shelves through the warehouse as effectively as possible. The second family keeps the same warehouse task and comparison axes, but changes the objective into a call-aware formulation that seeks to maintain delivery performance while reducing

unnecessary LM queries. In this family, the output interface includes a Steps field, allowing a selected action to persist for multiple simulation steps when repeated replanning is not useful. The third family extends the same framework toward battery-aware control, where the purpose is to study whether the controller can incorporate resource constraints in addition to throughput and call efficiency. Together, these objective families make it possible to compare not only absolute delivery performance, but also how the same supervisory loop behaves when the definition of a good decision changes.

Two planning architectures are compared inside this framework. In the centralized architecture, one LM call plans for all agents that currently need a decision, using a prompt that exposes the global state and the relevant candidate actions. This gives the model an explicit opportunity to coordinate joint behaviour, but it also increases the prompt size and output complexity as the number of agents and candidate actions grows. In the shared-context per-agent architecture, the model plans for one agent at a time. Each prompt still contains global information, but the required output concerns only the current agent. This reduces parsing burden and makes the decision format simpler, while shifting coordination from one explicit joint plan to a sequence of local decisions made with shared context. The comparison is therefore meaningful because it separates two practical ways of embedding language models in multi-agent control: broad joint reasoning versus simpler repeated decisions.

Prompt representation is treated as another comparison axis rather than as a separate experiment in isolation. The prompts always ground the model’s choice in explicit action ids and role-relevant state, including requested-shelf mappings, rules for AGVs and pickers, support relations such as waiting-for-picker context, battery-state abstractions, and filtered or prioritized candidate actions. What varies is the form in which the model is asked to respond, especially natural-language action selection versus structured JSON output. This comparison tests whether stronger output structure improves reliability without changing the underlying warehouse task. Taken together, the principal comparison axes are model size, centralized versus shared-context per-agent planning, natural-language versus JSON prompting, and the three objective formulations of delivery-only, call-aware delivery, and battery-aware control.

3.5 Scenarios, Models, and Metrics

The same scenario catalogue is reused across the experiment families so that differences in observed behaviour can be interpreted against known coordination pressures rather than against unrelated task changes. The scenarios are `tiny_balanced_1v1`, `small_balanced_2v2`, `medium_balanced_4v4`, `small_agv_heavy_4v2`, `medium_agv_heavy_6v2`, and `small_picker_heavy_2v4`. These scenarios cover simple balanced collaboration, in-

creased concurrency, larger warehouse layouts, AGV-heavy settings where picker support can become a bottleneck, and picker-heavy settings where allocation efficiency and underutilization become more visible. The scenario set therefore tests whether a model, prompt form, or planning architecture remains productive when the source of coordination difficulty changes.

The model set spans smaller and larger local models: llama3.2:1b, llama3.2:3b, mistral:7b, gemma3:12b, phi4, and qwen2.5:14b. Using models of different sizes supports the thesis question about whether larger models produce better coordination, or whether smaller models can remain competitive when the prompt is sufficiently grounded and the validation layer prevents infeasible actions from being executed. Because all models operate through the same discrete-action interface, the comparison concerns decision quality within the same controller framework rather than differences in low-level robot control.

The primary metric is delivery throughput, expressed as pick rate, which is computed from the number of delivered shelves and the simulated episode duration. Total shelf deliveries are also reported directly because they are easy to interpret in relation to the task objective and the episode-level delivery ceiling. Additional metrics are collected to explain how a controller reaches its throughput outcome: clashes, deadlocks or stuck events, distance travelled by each agent type, episode return, episode length, and inference speed or latency when measured. These supporting metrics are necessary because two configurations can achieve similar delivery counts for different reasons. For example, one configuration may maintain throughput with fewer model calls, while another may show the same delivery count but with more clashes, longer no-delivery tails, or inefficient movement. The metrics therefore connect the numerical result to the underlying collaborative behaviour.

3.6 Experimental Protocol

The reported experiment sessions were run with fixed environment ids and a fixed seed, creating a controlled set of comparable runs for the reported objective families. Within that controlled setup, the campaign proceeds from Objective 1 delivery-only experiments to Objective 2 call-aware delivery experiments and then to Objective 3 battery-aware experiments. This ordering is deliberate. The delivery-only experiments first establish how the models, prompt forms, and planning architectures behave when the objective is simply to complete warehouse deliveries. The call-aware experiments then ask whether the same framework can reduce unnecessary replanning without losing the productive behaviour established in the first family. The battery-aware experiments extend the evaluation to resource-sensitive decision making, where the controller must

consider battery constraints as part of the same high-level assignment problem.

Section 4 reports Objective 1 for the shelf-delivery comparisons, Objective 2 for the call-aware comparisons, and Objective 3 for the battery-aware objective. In the current simulation setup, a run can achieve at most 20 deliveries, and the results chapter uses that ceiling to contextualize reported scores.

3.7 Delimitations

To keep the study focused and interpretable, the following delimitations apply:

- The work is evaluated in a simulated warehouse environment rather than on physical robots.
- The controller is responsible only for high-level target selection; low-level navigation and collision handling are performed by the environment.
- The evaluated language models are prompted general-purpose models accessed through a local inference server and are not fine-tuned for this task.
- The reported experiments use fixed seeds and a limited number of completed sessions.
- Completed aggregate results are not yet available for the battery-aware objective family.

3.8 Ethical and Sustainability Considerations

Although the present work is simulation-based, it raises practical issues that remain relevant for real multi-robot deployments. The primary ethical concern is safety and reliability: generated decisions may be infeasible, unstable, or unproductive if they are not constrained by system-side checks. For that reason, the studied control loop does not execute free-form outputs directly. It uses grounded action identifiers, candidate shaping, explicit feasibility validation, and fallback behavior so that unsafe or clearly invalid actions are filtered before execution. The logging of prompts, parsed outputs, executed actions, and observed step consequences also improves traceability, which is important when analyzing failures such as long no-delivery tails, stale waiting loops, or battery-related breakdowns.

From a sustainability perspective, the main trade-off is between model capability and computational cost. Larger language models generally require more inference time and compute, while smaller models may require stronger prompt structure and validation to remain effective. This thesis therefore treats call efficiency, unnecessary replanning, and coordination overhead as practically relevant concerns rather than only implementation details. Even with safeguards, fixed-seed evaluation, limited scenario coverage, and the absence of physical deployment still bound the scope of the reported results.

4 Preliminary Results and Discussion

4.1 Result Scope and Objective Progression

This chapter reports the available results for the three objective families. Objective 1 focuses on maximizing total shelf deliveries. Objective 2 retains shelf delivery as the task outcome, but changes the optimization target to improving delivery while reducing unnecessary LM calls. Objective 3 is defined as a battery-aware objective, but completed aggregate results are not yet available for that family.

For interpreting the reported numbers, one additional fact is important: in the current simulation setup, 20 deliveries is the maximum achievable total for a run. Reported delivery counts should therefore be read relative to this ceiling. A run with only a few deliveries is not merely weak in absolute terms; it is far below what the environment allows when coordination succeeds.

4.2 Objective 1: Shelf-Delivery Experiments

Objective 1 evaluates planning architecture and prompt form under the shelf-delivery objective.

4.2.1 Planning architecture

The architecture comparison in Objective 1 is summarized in Table 1.

Experiment family	Runs	Total deliveries	Average deliveries/run
Centralized + natural	36	15	0.42
Centralized + JSON	36	21	0.58
Shared-context + natural	36	25	0.69
Shared-context + JSON	36	0	0.00

Table 1 Aggregate outcome of the Objective 1 shelf-delivery experiments across models and scenarios.

Table 2 reports the matched natural-language Objective 1 runs, where the prompt form and objective are the same and the planning architecture differs.

Architecture	Runs	Total deliveries	Average deliveries/run
Centralized + natural	36	15	0.42
Shared-context + natural	36	25	0.69

Table 2 Direct Objective 1 architecture comparison under the matched natural-language setting.

Under this matched setting, shared-context planning produced 25 deliveries in aggregate and 0.69 deliveries per run on average, whereas centralized planning produced 15 deliveries in aggregate and 0.42 deliveries per run on average.

4.2.2 Prompt representation

Objective 1 and Objective 2 each contain matched shared-context runs with natural-language and JSON prompts. Table 3 reports those side-by-side comparisons.

Objective family	Architecture	Prompt form	Runs	Total deliveries
Objective 1	Shared-context	Natural language	36	25
Objective 1	Shared-context	JSON	36	0
Objective 2	Shared-context	Natural language	36	303
Objective 2	Shared-context	JSON	36	53

Table 3 Direct prompt-format comparisons within matched shared-context experiment families.

In Objective 1, the matched shared-context natural-language runs produced 25 deliveries, whereas the matched shared-context JSON runs produced zero. In Objective 2, the matched natural-language shared-context session produced 303 deliveries, whereas the matched JSON session produced 53.

4.3 Objective 2: Call-Aware Shelf-Delivery Experiments

Objective 2 changes the goal from pure delivery throughput to a joint objective: preserve delivery performance while reducing unnecessary LM calls. The output interface for this objective includes both an action identifier and a Steps value, where Steps specifies how long the chosen action should remain active before the next query.

Across 36 runs, the completed Objective 2 natural-language shared-context session produced 303 deliveries, corresponding to an average of 8.42 deliveries per run.

4.3.1 Prompt representation

The completed Objective 2 sessions provide a direct prompt-format comparison within the shared-context experiment family. The natural-language session produced 303 deliveries across 36 runs, corresponding to 8.42 deliveries per run. The completed JSON session produced 53 deliveries across the same number of runs, corresponding to 1.47 deliveries per run. Efficiency measures show the same pattern: the natural-language session required approximately 204.7 LM calls per delivery and 469.5 seconds per delivery, while the JSON session required approximately 1078.4 calls per delivery and 3057.4 seconds per delivery.

The Objective 2 JSON session produced non-zero throughput, with deliveries reported for gemma3:12b and phi4. The natural-language session had 2 zero-delivery runs out of 36, whereas the JSON session had 26 zero-delivery runs out of 36.

4.3.2 Adaptation to the changed objective

Objective 2 reports the available data for the call-aware objective. The completed runs report delivery totals together with call-related metrics through the Steps output and the logged query counts.

4.3.3 Model size

Table 4 reports the completed model-size comparison from Objective 2.

Model	Total deliveries	Average deliveries/scenario	Seconds/delivery
llama3.2:3b	73	12.17	238.5
gemma3:12b	63	10.50	460.2
mistral:7b	59	9.83	530.5
qwen2.5:14b	46	7.67	495.6
llama3.2:1b	36	6.00	371.4
phi4	26	4.33	1091.6

Table 4 Aggregate results for the Objective 2 shared-context natural-language session.

Within the completed natural-language session, llama3.2:3b produced the highest total delivery count and the lowest seconds-per-delivery value among the listed models. In the completed JSON session, gemma3:12b and phi4 produced 26 and 25 deliveries

respectively, while both llama3.2 variants produced zero deliveries across all six scenarios.

4.3.4 Call-efficiency outcome

Objective 2 reports both throughput and LM usage. Raw calls-per-delivery were approximately 146.2 for phi4, 158.3 for gemma3:12b, 167.0 for qwen2.5:14b, 167.7 for llama3.2:3b, 287.0 for mistral:7b, and 316.3 for llama3.2:1b.

4.4 Objective 3: Battery-Aware Objective

Objective 3 defines a battery-aware objective. Completed aggregate results are not yet available for this objective family.

4.5 Discussion

4.5.1 RQ1: Model size

The available data for RQ1 come mainly from Objective 2. Table 4 reports the completed model-size comparison for the shared-context natural-language session, and the accompanying text reports the corresponding JSON outcomes where available.

4.5.2 RQ2: Planning architecture

The available data for RQ2 come from the matched Objective 1 natural-language comparison, where shared-context planning produced 25 deliveries and centralized planning produced 15.

4.5.3 RQ3: Prompt representation

The available data for RQ3 come from the matched shared-context prompt-format comparisons in Objective 1 and Objective 2. These comparisons report delivery totals, zero-delivery runs, calls per delivery, and seconds per delivery for natural-language and JSON prompting.

The reported sessions use fixed seeds. Completed aggregate results are not yet available for Objective 3.

5 Conclusions and Future Work

This thesis evaluates LM-driven decision-making for heterogeneous multi-robot collaboration in a task-assignment warehouse setting with battery constraints. The implemented control loop grounds decisions to discrete environment action identifiers, queries an LM using centralized or per-agent shared-context prompts, and enforces feasibility through action-mask validation.

The reported results cover Objective 1 and Objective 2. Objective 1 provides the current data for planning-architecture and prompt-format comparisons under the shelf-delivery objective. Objective 2 provides the current data for call-aware control, including delivery totals, calls per delivery, seconds per delivery, and the model-size comparison for the shared-context natural-language session.

Within the available data, Objective 1 reports more deliveries for shared-context planning than for centralized planning in the matched natural-language comparison, and more deliveries for natural-language prompting than for JSON prompting in the matched shared-context comparison. Objective 2 reports 303 deliveries for the completed natural-language shared-context session and 53 deliveries for the completed JSON shared-context session. In the completed natural-language model comparison for Objective 2, `llama3.2:3b` produced the highest total delivery count.

Completed aggregate results are not yet available for Objective 3, the battery-aware objective. This objective remains part of the defined evaluation framework, but its aggregate outcome is not yet reported in the current results chapter.

5.1 Future Work

- **Structured outputs and tighter parsing:** require the model to output a strict schema (e.g., `{action_id, hold_steps, rationale}`) and reject non-conforming responses to reduce ambiguity.
- **Centralized versus decentralized planning:** compare per-agent prompts (current approach) with a centralized planner that outputs a joint action vector, including explicit coordination constraints.
- **Candidate-set design:** learn or compute compact candidate action sets (e.g., nearest requested shelves, relevant support targets for pickers) while guaranteeing that necessary actions remain available.

- **Multi-seed and scaling studies:** extend evaluation to many random seeds and larger layouts (more shelves, more agents) to quantify robustness and scalability.
- **MARL baselines:** train/evaluate proximal policy optimization (PPO) or other MARL methods on Battery-TA-RWARE and compare to LM in throughput, generalization, and compute cost.
- **Cost-aware deployment:** measure inference latency, token usage, and energy cost across model sizes to assess feasibility for embedded or real-time settings.

References

- [1] M. Tomasello, *Origins of Human Communication*. Cambridge, MA, USA: MIT Press, 2008.
- [2] R. Dunbar, *Grooming, Gossip, and the Evolution of Language*. Cambridge, MA, USA: Harvard University Press, 1996.
- [3] H. H. Clark, *Using Language*. Cambridge, UK: Cambridge University Press, 1996.
- [4] S. Johansson, *Origins of Language: Constraints on Hypotheses*, ser. Converging Evidence in Language and Communication Research. Amsterdam, The Netherlands and Philadelphia, PA, USA: John Benjamins Publishing Company, 2005, vol. 5.
- [5] S. R. Fischer, *A History of Writing*. London, UK: Reaktion Books, 2004.
- [6] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3rd ed. Draft manuscript, 2025, draft version. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>
- [7] Y. Bengio, R. Ducharme, P. Vincent, and C. Janvin, “A neural probabilistic language model,” *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003. [Online]. Available: <https://jmlr.org/papers/v3/bengio03a.html>
- [8] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need>
- [10] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, E. Brynjolfsson, S. Buch, D. Card, R. Castellon, N. Chatterji, A. Chen, K. Creel, J. Q. Davis, D. Demszky, C. Donahue, M. Doumbouya, E. Durmus, S. Ermon, J. Etchemendy, K. Ethayarajh, L. Fei-Fei, C. Finn, T. Gale, L. Gillespie, K. Goel, N. Goodman, S. Grossman, N. Guha, T. Hashimoto, P. Henderson, J. Hewitt, D. E. Ho, J. Hong, K. Hsu, J. Huang, T. Icard, S. Jain, D. Jurafsky, P. Kalluri, S. Karamcheti, G. Keeling,

- F. Khani, O. Khattab, P. W. Koh, M. Krass, R. Krishna, R. Kuditipudi, A. Kumar, F. Ladhak, M. Lee, T. Lee, J. Leskovec, I. Levent, X. L. Li, X. Li, T. Ma, A. Malik, C. D. Manning, S. Mirchandani, E. Mitchell, Z. Munyikwa, S. Nair, A. Narayan, D. Narayanan, B. Newman, A. Nie, J. C. Niebles, H. Nilforoshan, J. Nyarko, G. Ogut, L. Orr, I. Papadimitriou, J. S. Park, C. Piech, E. Portelance, C. Potts, A. Raghunathan, R. Reich, H. Ren, F. Rong, Y. Roohani, C. Ruiz, J. Ryan, C. Ré, D. Sadigh, S. Sagawa, K. Santhanam, A. Shih, K. Srinivasan, A. Tamkin, R. Taori, A. W. Thomas, F. Tramèr, R. E. Wang, W. Wang, B. Wu, J. Wu, Y. Wu, S. M. Xie, M. Yasunaga, J. You, M. Zaharia, M. Zhang, T. Zhang, X. Zhang, Y. Zhang, L. Zheng, K. Zhou, and P. Liang, “On the opportunities and risks of foundation models,” *arXiv preprint arXiv:2108.07258*, 2021.
- [11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [12] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” *arXiv preprint arXiv:2201.07207*, 2022.
- [13] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 2609–2634. [Online]. Available: <https://aclanthology.org/2023.acl-long.147/>
- [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [15] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract.html
- [16] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

-
- [17] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.
- [18] Z. Yan, N. Jouandeau, and A. A. Cherif, “A survey and analysis of multi-robot coordination,” *International Journal of Advanced Robotic Systems*, vol. 10, no. 12, 2013, accessed 2026-02-26. [Online]. Available: <https://journals.sagepub.com/doi/full/10.5772/57313>
- [19] Statista, “E-commerce market data and statistics,” 2025, global retail e-commerce sales estimates.
- [20] UNCTAD, “Digital economy report 2024,” 2024.
- [21] J. J. Bartholdi, III and S. T. Hackman, *Warehouse & Distribution Science*. Atlanta, GA, USA: Georgia Institute of Technology, Aug. 2019, release 0.98.1. [Online]. Available: <https://www.warehouse-science.com/book/editions/wh-sci-0.98.1.pdf>
- [22] R. de Koster, T. Le-Duc, and K. J. Roodbergen, “Design and control of warehouse order picking: A literature review,” *European Journal of Operational Research*, vol. 182, no. 2, pp. 481–501, 2007.
- [23] N. Boysen, R. de Koster, and F. Weidinger, “Warehousing in the e-commerce era: A survey,” *European Journal of Operational Research*, vol. 277, no. 2, pp. 396–411, 2019.
- [24] Í. R. d. C. Barros and T. P. d. Nascimento, “Robotic mobile fulfillment systems: A survey on recent developments and research opportunities,” *Robotics and Autonomous Systems*, vol. 137, p. 103729, 2021.
- [25] P. R. Wurman, R. D’Andrea, and M. Mountz, “Coordinating hundreds of cooperative, autonomous vehicles in warehouses,” *AI Magazine*, vol. 29, no. 1, pp. 9–20, 2008.
- [26] R. Karam, A. A. Nguyen, R. Lin, D. R. Martin, D. Morales, B. A. Butler, and M. Egerstedt, “Collaboration in multi-robot systems: Taxonomy and survey over frameworks for collaboration,” 2026.
- [27] G. Papoudakis, F. Christianos, L. Schäfer, and S. V. Albrecht, “Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks,” 2021, accessed 2026-02-26. [Online]. Available: <https://arxiv.org/abs/2006.07869>
- [28] A. Krnjaic, R. D. Steleac, J. D. Thomas, G. Papoudakis, L. Schäfer, A. W. K. To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Börsting, and S. V. Albrecht, “Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 677–684.

-
- [29] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “OpenAI Gym,” *arXiv preprint arXiv:1606.01540*, 2016.
- [30] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. T. J. Shen, and O. G. Younis, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.
- [31] B. P. Gerkey and M. J. Matarić, “A formal analysis and taxonomy of task allocation in multi-robot systems,” *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.
- [32] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, “Conflict-based search for optimal multi-agent pathfinding,” *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.
- [33] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [34] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” *arXiv preprint arXiv:2209.07753*, 2022.
- [35] Z. Mandi, S. Jain, and S. Song, “Roco: Dialectic multi-robot collaboration with large language models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 286–299, accessed 2026-02-26. [Online]. Available: <https://ieeexplore.ieee.org/document/10610855>
- [36] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. K. Chadha, “A systematic survey of prompt engineering in large language models: Techniques and applications,” *ACM Computing Surveys*, 2024.
- [37] Z. R. Tam, A. H. Kargaran, D. Khyani, T. Gerstenberg, and H. Kämper, “Let me speak freely? a study on the impact of format restrictions on performance of large language models,” *arXiv preprint arXiv:2408.02442*, 2024.
- [38] X. Niu and D. G. Broo, “Investigating symbiosis in robotic ecosystems: A case study for multi-robot reinforcement learning reward shaping,” in *2025 9th International Conference on Robotics and Automation Sciences (ICRAS)*, 2025, pp. 112–117.

- [39] X. Niu, N. C. Barajas, and D. G. Broo, “Enabling symbiosis in multi-robot systems through multi-agent reinforcement learning,” in *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–7.
- [40] P. Li, Z. An, S. Abrar, and L. Zhou, “Large language models for multi-robot systems: A survey,” 2026. [Online]. Available: <https://arxiv.org/abs/2502.03814>
- [41] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” *arXiv preprint arXiv:2207.05608*, 2022.
- [42] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “Progprompt: Generating situated robot task plans using large language models,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [43] Y. U. Cao, A. S. Fukunaga, and A. B. Kahng, “Cooperative mobile robotics: Antecedents and directions,” *Autonomous Robots*, vol. 4, no. 1, pp. 7–27, 1997.
- [44] T. Arai, E. Pagello, and L. E. Parker, “Editorial: Advances in multi-robot systems,” *IEEE Transactions on Robotics and Automation*, vol. 18, no. 5, pp. 655–661, 2002.
- [45] M. A. Sebok and H. G. Tanner, “Cooperative planning for physically interacting heterogeneous robots,” *Frontiers in Robotics and AI*, vol. 11, p. 1172105, 2024. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/frobt.2024.1172105/full>
- [46] C. Zhang and V. Lesser, “Coordinating multi-agent reinforcement learning with limited communication,” in *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2013, accessed 2026-02-26. [Online]. Available: <https://www.ifaamas.org/Proceedings/aamas2013/docs/p1101.pdf>
- [47] V. Bhat, A. U. Kaypak, P. Krishnamurthy, R. Karri, and F. Khorrami, “Grounding large language models for robot task planning using closed-loop state feedback,” 2025, received: 27 April 2025; revised: 10 October 2025; accepted: 17 October 2025.
- [48] Y. Huang, “Collaborative multi-robot task planning using large language models,” Master’s thesis, Uppsala University, 2025, accessed 2026-02-26. [Online]. Available: <https://uu.diva-portal.org/smash/get/diva2:2023514/FULLTEXT01.pdf>
- [49] A. Dawane, “On-device small language model inference acceleration for real-time robotics: A survey,” in *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, 2026.

- [50] L. Xiao and T. Yamasaki, “Probing prompt design for socially compliant robot navigation with vision-language models,” 2026, accessed 2026-02-26.

A Appendix

A.1 Reproducibility: Running the Controllers

The experiments in this thesis are executed from the Battery-TA-RWARE repository in this workspace. Example commands:

```
1 python Battery-TA-RWARE/scripts/run_llm.py \
2   --env_id tarware-tiny-lagvs-lpickers-partialobs-chg-v1 \
3   --seed 0 \
4   --num_episodes 1 \
5   --model llama3.2:3b
```

Listing 1: Run the basic per-agent language-model controller for one episode.

```
1 python Battery-TA-RWARE/scripts/run_centralized_llm_experiments.py \
2   --prompt_format language \
3   --seed 0
```

Listing 2: Run the centralized shelf-delivery experiments.

```
1 python Battery-TA-RWARE/scripts/run_shared_context_llm_expt.py \
2   --prompt_format language \
3   --seed 0
```

Listing 3: Run the shared-context shelf-delivery experiments.

```
1 python Battery-TA-RWARE/scripts/run_obj2_shared_context_llm.py \
2   --prompt_format language \
3   --max_steps 0 \
4   --max_busy_steps_for_replan 10 \
5   --max_inactivity_steps 500 \
6   --disable_support_needed_soon \
7   --find_path_agent_aware_always \
8   --seed 0
```

Listing 4: Run the Objective 2 shared-context call-aware experiments.

```
1 python Battery-TA-RWARE/scripts/run_heuristic.py \
2   --env_id tarware-tiny-lagvs-lpickers-partialobs-chg-v1 \
3   --seed 0 \
4   --num_episodes 1
```

Listing 5: Run the heuristic policy for one episode.

The main result folders discussed in the thesis are:

- results/shelf/central_llm_experiments: Objective 1, centralized planning.
- results/shelf/shared_context_llm_experiments: Objective 1, shared-context per-agent planning.
- results/calls/shared_context: Objective 2, shared-context per-agent planning with call-efficiency objective.
- results/battery: planned battery-oriented experiments (not yet populated with completed aggregate sessions at the time of writing).

A.2 Prompt Excerpt (Illustrative)

The LM is prompted with an explicit mapping from relevant state elements to discrete action identifiers. The exact prompt depends on the experiment family: centralized prompts describe all agents at once, while shared-context prompts describe one current agent together with selected global context. The full prompts are generated programmatically; the excerpt below illustrates the style (line breaks and ordering may vary):

```

1 Current global state:
2 - Requested shelves:
3   - Requested shelf 24 is at position (11,5). To go there, choose
     action id 50.
4   - Requested shelf 39 is at position (6,8). To go there, choose
     action id 29.
5 ...
6 You are agent_0 (type=AGV).
7 Choose exactly one action_id from the candidate list.
8 Output format: action_id=<integer>

```

Listing 6: Illustrative prompt excerpt.

A.3 Metric Definitions

- **Total shelf deliveries:** number of requested shelves delivered to goal locations during an episode.
- **Overall pick rate:** deliveries/hour computed as

$$\text{pick_rate} = \frac{\text{deliveries} \cdot 3600}{5 \cdot \text{episode_steps}},$$

where 5 steps/second is the environment’s time scale used by the logging script.

- **Clashes and stuck events:** environment-defined counts related to collision resolution and deadlock-like behaviour.
- **Global episode return:** sum of environment rewards across all agents and time steps.